

Pintos Project 4 Design Document

```
+-----+
|      CS 140      |
| PROJECT 4: FILE SYSTEMS |
|    DESIGN DOCUMENT    |
+-----+
```

---- GROUP ----

>> Fill in the names and email addresses of your group members.

Jeonghoon Park <hoonably@unist.ac.kr>
Deokhyeon Kim <ejrgus1404@unist.ac.kr>

---- PRELIMINARIES ----

>> If you have any preliminary comments on your submission, notes for the
>> TAs, or extra credit, please give them here.

>> Please cite any offline or online sources you consulted while
>> preparing your submission, other than the Pintos documentation, course
>> text, lecture notes, and course staff.

INDEXED AND EXTENSIBLE FILES
=====

---- DATA STRUCTURES ----

>> A1: Copy here the declaration of each new or changed 'struct' or
>> 'struct' member, global or static variable, 'typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

```
#define INODE_DIRECT_CNT 125  => Number of direct data-block pointers in on-disk inode (4 bytes * 125 = 500 bytes)
#define INDIRECT_BLOCK_CNT 128 => Number of sector pointers per single-level indirect block (512 bytes / 4 bytes = 128)

struct inode_disk {
    off_t length;                => File size in bytes
    unsigned magic;              => Magic number for validation
    block_sector_t direct_block[INODE_DIRECT_CNT]; => Array of 125 direct data-block sector numbers
    block_sector_t indirect_block_sector;          => Sector number of a single-level indirect block that holds 128
sector entries
};
```

>> A2: What is the maximum size of a file supported by your inode
>> structure? Show your work.

```
Direct blocks:      125 * 512 bytes = 64,000 bytes
Single-indirect blocks: 128 * 512 bytes = 65,536 bytes
Total blocks:      (125 + 128) blocks = 253 blocks
Maximum file size:  253 * 512 bytes = 129,536 bytes
```

---- SYNCHRONIZATION ----

>> A3: Explain how your code avoids a race if two processes attempt to
>> extend a file at the same time.

To prevent races when two threads extend the same file concurrently,
we serialize all growth operations on an inode using the global
filesys_lock.

Both file_write() and its helper inode_write_at() acquire
lock_acquire(&filesys_lock) before:

1. Reading the inode's current length.
2. Calculating and allocating any new data blocks (direct and indirect).
3. Updating the on-disk length in the inode.
4. Flushing the updated inode metadata to disk.

Holding filesys_lock across these steps ensures that no two extenders
interleave block allocation or length updates. Each extension therefore
sees a consistent file size and allocates nonoverlapping blocks.

```
>> A4: Suppose processes A and B both have file F open, both
>> positioned at end-of-file. If A reads and B writes F at the same
>> time, A may read all, part, or none of what B writes. However, A
>> may not read data other than what B writes, e.g. if B writes
>> nonzero data, A is not allowed to see all zeros. Explain how your
>> code avoids this race.
```

Reader and writer I/O are serialized at sector granularity
by the same locking and block-I/O scheme:

1. Both `inode_read_at()` and `inode_write_at()` acquire `filesystem_lock` (or the buffer-cache's per-block lock) before performing any block operations.
2. Writes use a read-modify-write on a "bounce" buffer:
 - a. Read the entire 512-byte sector into a buffer.
 - b. Update only the target bytes in that buffer.
 - c. Issue one atomic `block_write()` for the whole sector.
3. Reads invoke a single atomic `block_read()` per sector.

Because each sector update is atomic and locked, a reader sees either the entire old sector or the entire new sector—never a mix or a zero-filled sector. This prevents A from observing any data other than what B actually wrote.

```
>> A5: Explain how your synchronization design provides "fairness".
>> File access is "fair" if readers cannot indefinitely block writers
>> or vice versa. That is, many processes reading from a file cannot
>> prevent forever another process from writing the file, and many
>> processes writing to a file cannot prevent another process forever
>> from reading the file.
```

Fairness is achieved because we use one short-grained exclusive lock for both reads and writes and hold it only during atomic sector operations. The Pintos semaphore backing the lock:

- Queues sleeping threads in arrival order when priorities tie.
- Wakes one waiting thread each time the lock is released.

Since each I/O or metadata update releases the lock quickly, readers cannot indefinitely block a writer, and writers cannot indefinitely block a reader. Both kinds of threads get eventual access when they wake up from the semaphore.

---- RATIONALE ----

```
>> A6: Is your inode structure a multilevel index? If so, why did you
>> choose this particular combination of direct, indirect, and doubly
>> indirect blocks? If not, why did you choose an alternative inode
>> structure, and what advantages and disadvantages does your
>> structure have, compared to a multilevel index?
```

Our inode uses a combined direct+single-indirect scheme, not full multilevel indexing. We originally considered double-indirect blocks but found them unnecessary—the seven test cases fit within one indirect level.

- Direct blocks (125 sector numbers): fast direct access for the first $125 \times 512 \text{ bytes} = 64,000 \text{ bytes}$ of file data.
- Single-indirect block (1 block of 128 sector entries): supports an additional $128 \times 512 \text{ bytes} = 65,536 \text{ bytes}$.
- Total capacity: $(125 + 128) \times 512 \text{ bytes} = 129,536 \text{ bytes}$.

This "mixed" design leverages unused inode space for more direct block references, preserving combined-scheme benefits without double-indirect complexity. Advantages: simpler code, fewer I/O steps for most files, and small metadata overhead. Disadvantages: lower maximum file size than if double-indirect were used, but adequate for project requirements.

SUBDIRECTORIES
=====

---- DATA STRUCTURES ----

```
>> B1: Copy here the declaration of each new or changed `struct' or
```

>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

---- ALGORITHMS ----

>> B2: Describe your code for traversing a user-specified path. How
>> do traversals of absolute and relative paths differ?

---- SYNCHRONIZATION ----

>> B4: How do you prevent races on directory entries? For example,
>> only one of two simultaneous attempts to remove a single file
>> should succeed, as should only one of two simultaneous attempts to
>> create a file with the same name, and so on.

>> B5: Does your implementation allow a directory to be removed if it
>> is open by a process or if it is in use as a process's current
>> working directory? If so, what happens to that process's future
>> file system operations? If not, how do you prevent it?

---- RATIONALE ----

>> B6: Explain why you chose to represent the current directory of a
>> process the way you did.

BUFFER CACHE
=====

---- DATA STRUCTURES ----

>> C1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

---- ALGORITHMS ----

>> C2: Describe how your cache replacement algorithm chooses a cache
>> block to evict.

>> C3: Describe your implementation of write-behind.

>> C4: Describe your implementation of read-ahead.

---- SYNCHRONIZATION ----

>> C5: When one process is actively reading or writing data in a
>> buffer cache block, how are other processes prevented from evicting
>> that block?

>> C6: During the eviction of a block from the cache, how are other
>> processes prevented from attempting to access the block?

---- RATIONALE ----

>> C7: Describe a file workload likely to benefit from buffer caching,
>> and workloads likely to benefit from read-ahead and write-behind.

SURVEY QUESTIONS
=====

Answering these questions is optional, but it will help us improve the
course in future quarters. Feel free to tell us anything you
want--these questions are just to spur your thoughts. You may also
choose to respond anonymously in the course evaluations at the end of
the quarter.

>> In your opinion, was this assignment, or any one of the three problems
>> in it, too easy or too hard? Did it take too long or too little time?

>> Did you find that working on a particular part of the assignment gave
>> you greater insight into some aspect of OS design?

>> Is there some particular fact or hint we should give students in
>> future quarters to help them solve the problems? Conversely, did you
>> find any of our guidance to be misleading?

>> Do you have any suggestions for the TAs to more effectively assist
>> students in future quarters?

>> Any other comments?

==== Peer Evaluation ====

+-----+-----+		
	Name	Contribution
+-----+-----+		
	Jeonghoon Park	50%, struct and overall inode layout design, direct block implementation
+-----+-----+		
	Deokhyeon Kim	50%, indirect block implementation (byte_to_sector, grow)
+-----+-----+		